

Тестирование PySpark-приложений

Цель видео

Научиться писать тесты для PySpark-приложений.

Пример создания локального SparkContext

```
from pyspark.sql import SparkSession

# Создание локального SparkSession для тестов
spark = SparkSession.builder \
    .appName("PySpark Testing") \
    .master("local[*]") \
    .getOrCreate()
```

Тестирование PySpark-приложений

Тестирование PySpark-приложений важно для обеспечения качества данных и корректности ETL-процессов.

Основные аспекты тестирования PySpark-приложений:

- Настройка окружения для тестирования
 - Создание локального SparkContext для тестов

Тестирование PySpark-приложений

Тестирование PySpark-приложений важно для обеспечения качества данных и корректности ETL-процессов.

Основные аспекты тестирования PySpark-приложений:

- Настройка окружения для тестирования
 - Создание локального SparkContext для тестов
- Библиотеки для тестирования
 - Установка PySpark и необходимых библиотек (unittest, pytest, pyspark.sql.Row)

Тестирование PySpark-приложений

Основные аспекты тестирования PySpark-приложений:

- Типы тестов
 - Юнит-тесты: проверка отдельных функций
 - Интеграционные тесты: проверка взаимодействия между компонентами
 - E2E-тесты (End-to-End): проверка всего процесса обработки данных от начала до конца

Пример теста для PySpark-приложения

Рассмотрим простой пример, где мы будем тестировать функцию, которая фильтрует строки в DataFrame:

```
from pyspark.sql import Row

def filter_data(df, threshold):

    return df.filter(df.value > threshold)
```

Пример теста для PySpark-приложения

```
class PySparkAppTests(PySparkTestCase):  
  
    def test_filter_data(self):  
  
        # Шаг 1: Подготавливаем тестовые данные  
  
        data = [Row(value=1), Row(value=10), Row(value=100)]  
  
        df = self.spark.createDataFrame(data)  
  
        # Шаг 2: Применяем тестируемую функцию  
  
        result_df = filter_data(df, 10)  
  
        # Шаг 3: Проверяем результаты  
  
        result = [row.value for row in result_df.collect()]  
  
        self.assertEqual(result, [100])
```


Объяснение примера

Функция для тестирования: `filter_data` принимает `DataFrame` и пороговое значение, фильтруя строки, в которых значение больше порога.

Создание теста:

- **Данные:** создание `DataFrame` с тестовыми данными
- **Фильтрация:** применение функции `filter_data`
- **Проверка результата:** сбор данных из результирующего `DataFrame` и сравнение их с ожидаемым результатом

Заключение

❶ Подготовка окружения

Прежде чем писать тесты, необходимо подготовить окружение для работы с PySpark. Устанавливайте необходимые библиотеки и создавайте SparkSession, который будет использоваться для тестов

Заключение

❶ Подготовка окружения

Прежде чем писать тесты, необходимо подготовить окружение для работы с PySpark. Устанавливайте необходимые библиотеки и создавайте SparkSession, который будет использоваться для тестов

❷ Создание тестового класса

Используйте unittest для организации тестов. Создайте базовый класс для тестов, который будет инициализировать и завершать SparkSession

Заключение

③ Написание тестов

Для каждой тестируемой функции создайте тестовый метод, где:

- **подготавливайте тестовые данные:** создайте DataFrame с тестовыми данными
- **применяйте тестируемую функцию:** вызывайте функцию, передавая ей DataFrame
- **проверяйте результаты:** сравнивайте результат работы функции с ожидаемым результатом

Заключение

3 Написание тестов

Для каждой тестируемой функции создайте тестовый метод, где:

- **подготавливайте тестовые данные:** создайте DataFrame с тестовыми данными
- **применяйте тестируемую функцию:** вызывайте функцию, передавая ей DataFrame
- **проверяйте результаты:** сравнивайте результат работы функции с ожидаемым результатом

4 Запуск тестов

Запускайте тесты с помощью unittest, чтобы убедиться, что все тесты проходят успешно